Minimum Spanning Tree

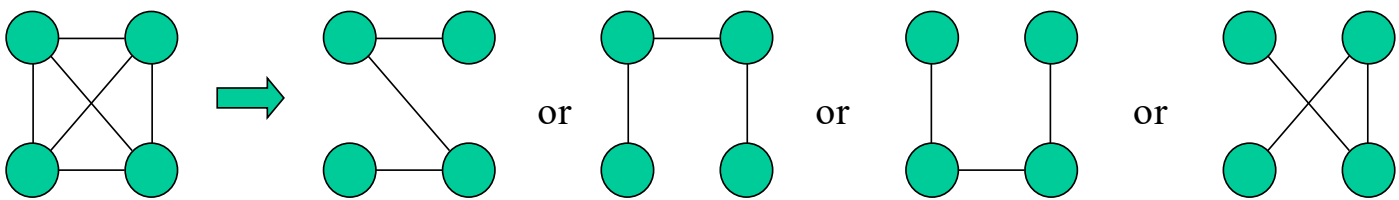
Spanning Trees

A spanning tree of a graph is just a subgraph that contains all the vertices and is a tree. without loops

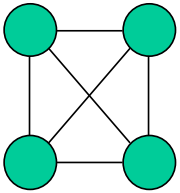
A graph may have many spanning trees.

Graph A

Some Spanning Trees from Graph A



Complete Graph

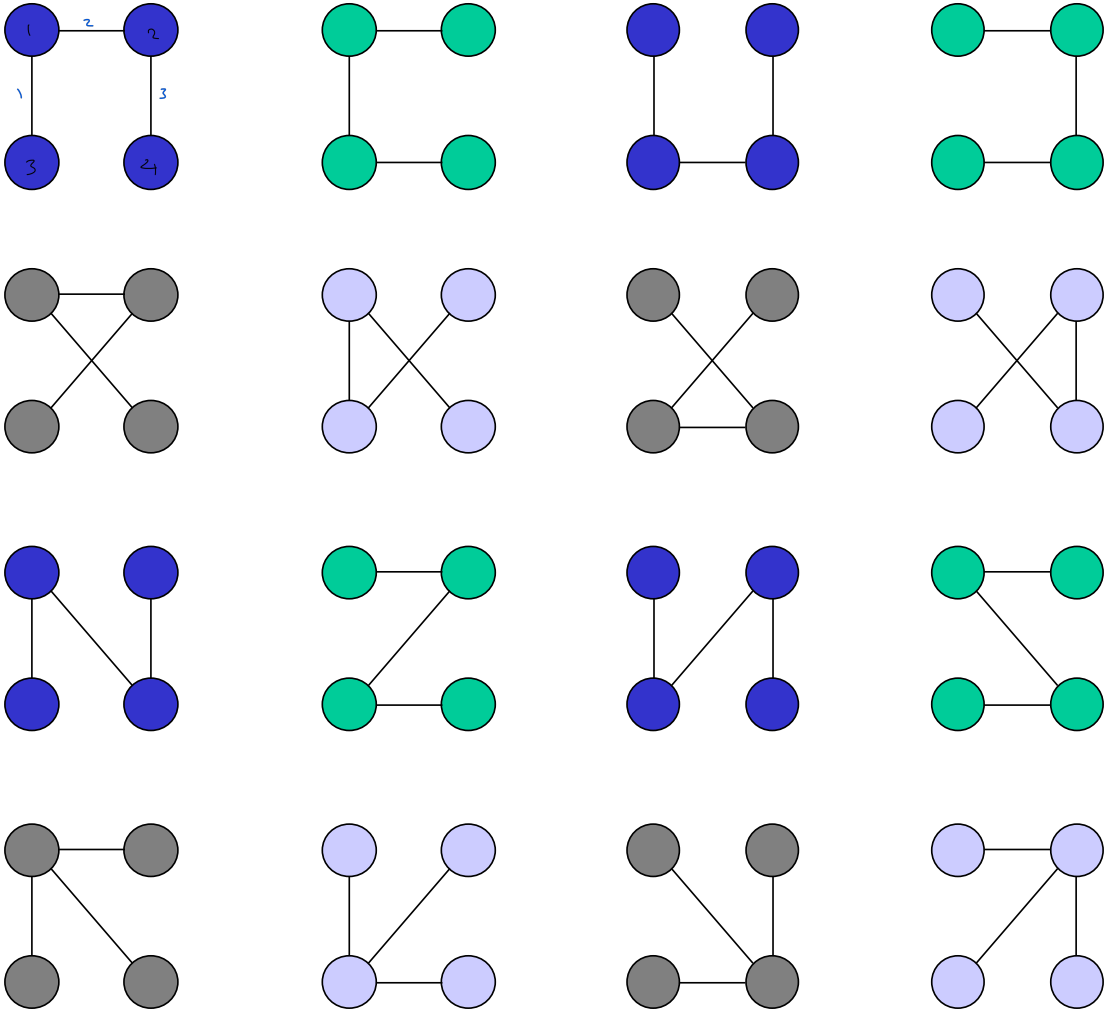


Number of edges = n-1
↓
vertices

Adding any more edge in any of the spanning tree makes a loop

4 vertices always has 3 edges

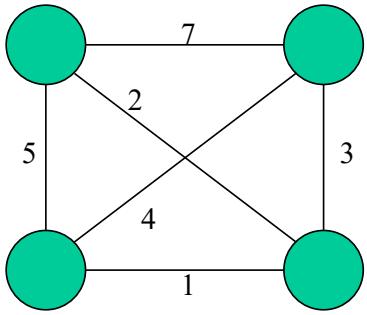
All 16 of its Spanning Trees



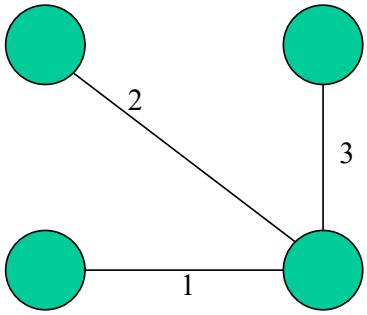
Minimum Spanning Trees

The Minimum Spanning Tree for a given graph is the Spanning Tree of minimum cost for that graph.

Complete Graph



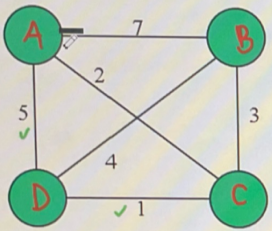
Minimum Spanning Tree



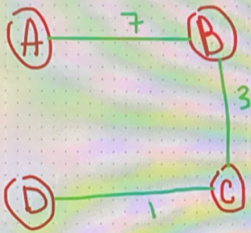
In some graphs, there could be more than one minimum spanning tree but this graph only has 1

Course: Design and Analysis of | Untitled – FigJam
figma.com/board/r3my01KiectRJUyw0gjiZx/Untitled?node-id=0-1&p=f&t=elxlHwhHe6q8l2jq-0
Verify that it's you
Untitled Free
Share

Complete Graph

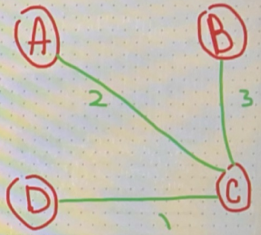


Spanning Tree



7 + 3 + 1 = 11

Minimum Spanning Tree



3 + 2 + 1 = 6

Search
ENG
9:21 AM
1/25/2026

Algorithms for Obtaining the Minimum Spanning Tree

- Kruskal's Algorithm
- Prim's Algorithm

Kruskal's Algorithm

Check the smallest numbers in the edge, if it doesn't make a loop, add it

If it asked about the algorithm, writing its name is usually enough but explaining it is better (when explaining it, u don't need to write the name?)

This algorithm creates a forest of trees. Initially the forest consists of n single node trees (and no edges). At each step, we add one edge (the cheapest one) so that it joins two trees together. If it were to form a cycle, it would simply link two nodes that were already part of a single connected tree, so that this edge would not be needed.

The steps are:

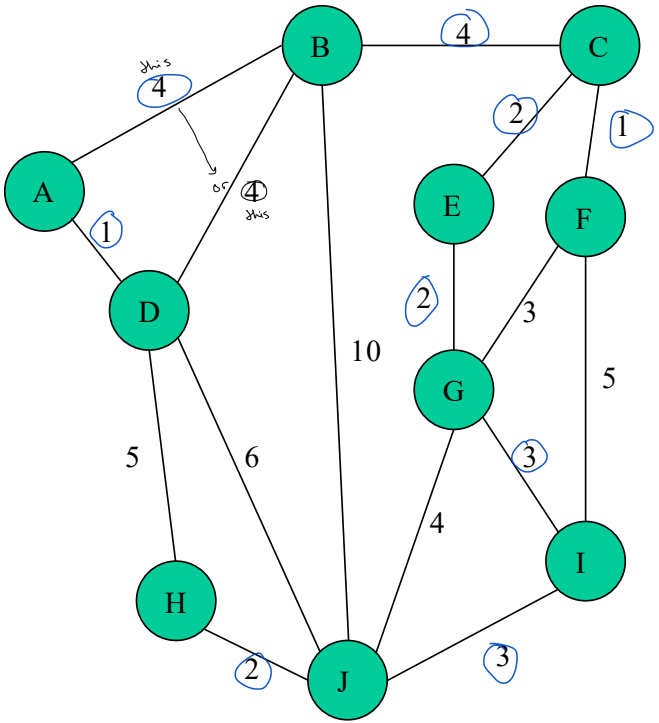
1. The forest is constructed - with each node in a separate tree.
2. The edges are placed in a priority queue.
3. Until we've added n-1 edges,
    1. Extract the cheapest edge from the queue,
    2. If it forms a cycle, reject it,
    3. Else add it to the forest. Adding it to the forest will join two trees together.

Every step will have joined two trees in the forest together, so that at the end, there will only be one tree in T.

Complete Graph

Start with 1 then 2 then 3 and so on.

If it makes a loop, reject it.



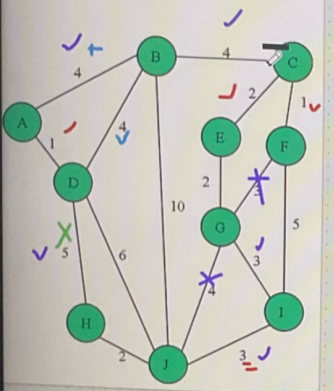
figma.com/board/LFFu8RWDnUrF9oj6GKsX1F/Untitled?node-id=0-1&p=f&t=elxlHwhHe6q8l2jq-0

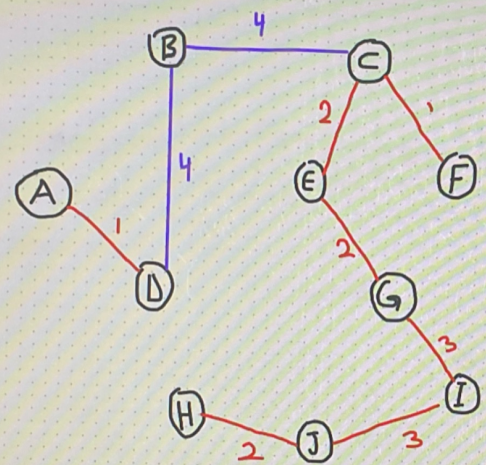
Untitled Free

Verify that it's you

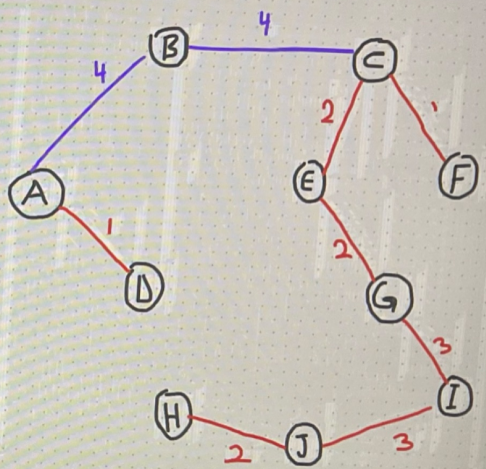
03:00 Share

Complete Graph





1+4+4+2+1+2+3+3+2 = 22



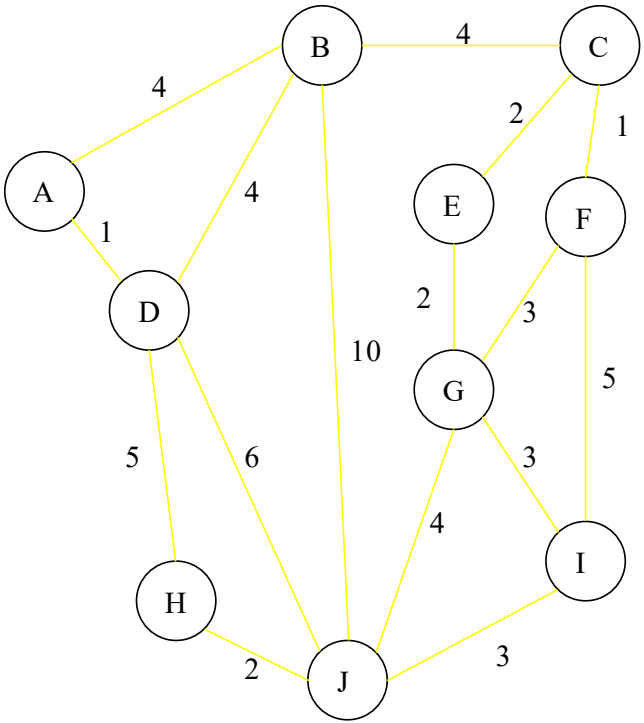
1+4+4+2+1+2+3+3+2 = 22

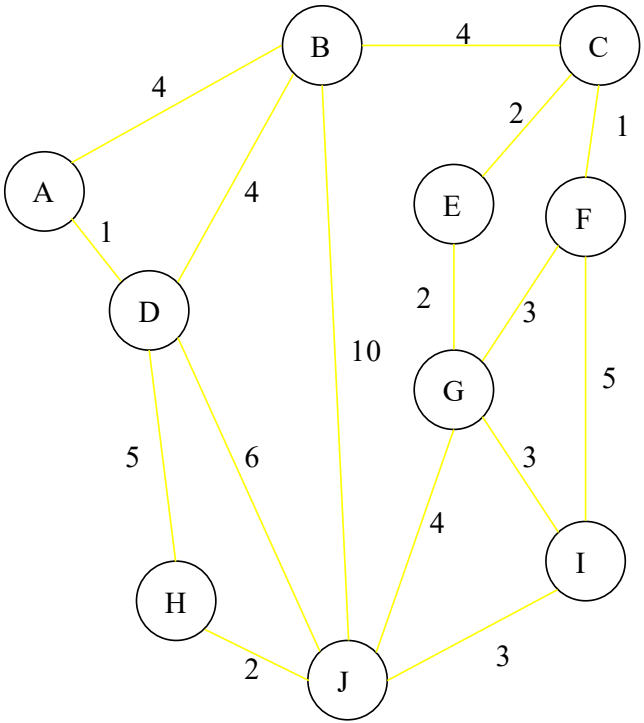
Same cost but different spanning tree

⇓

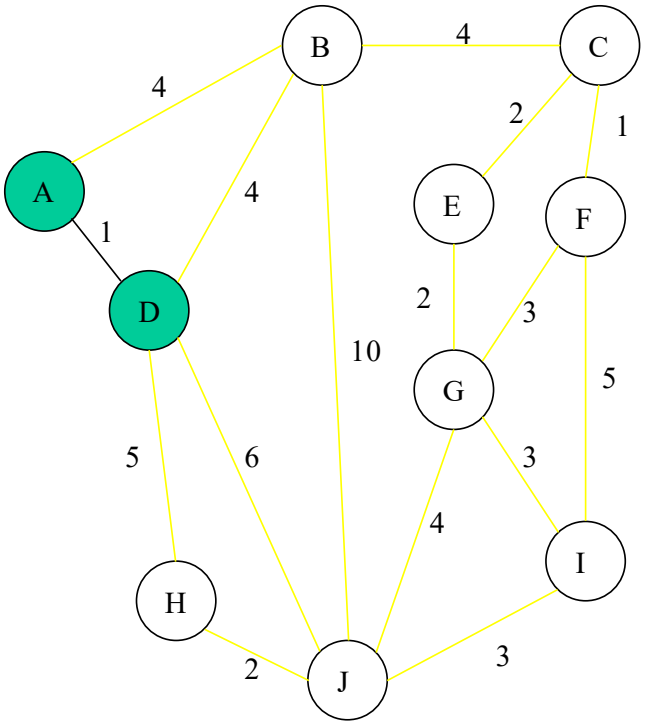
both are correct

Search

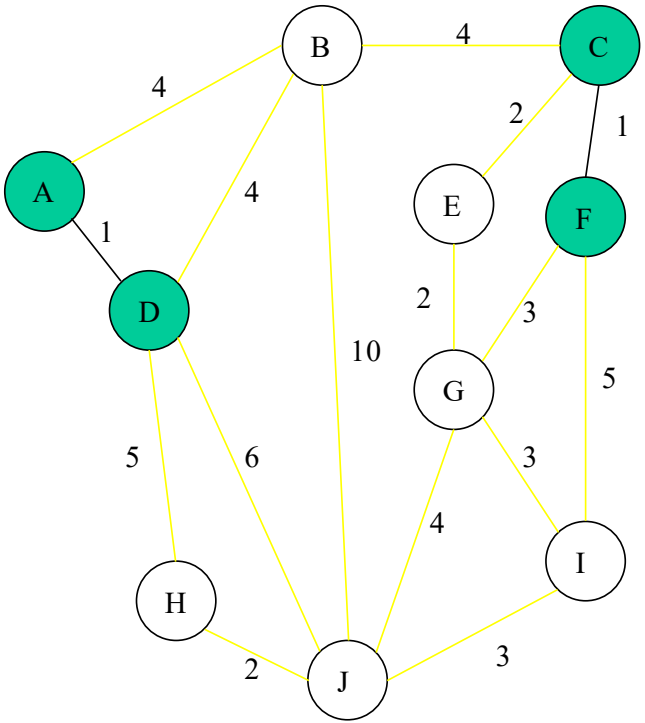

A
B
C
D
E
F
G
H
I
J
4
4
4
1
2
1
2
3
10
5
5
6
3
4
2
3


A
B
C
D
E
F
G
H
I
J
4
4
4
1
2
1
2
3
10
5
5
6
3
4
2
3

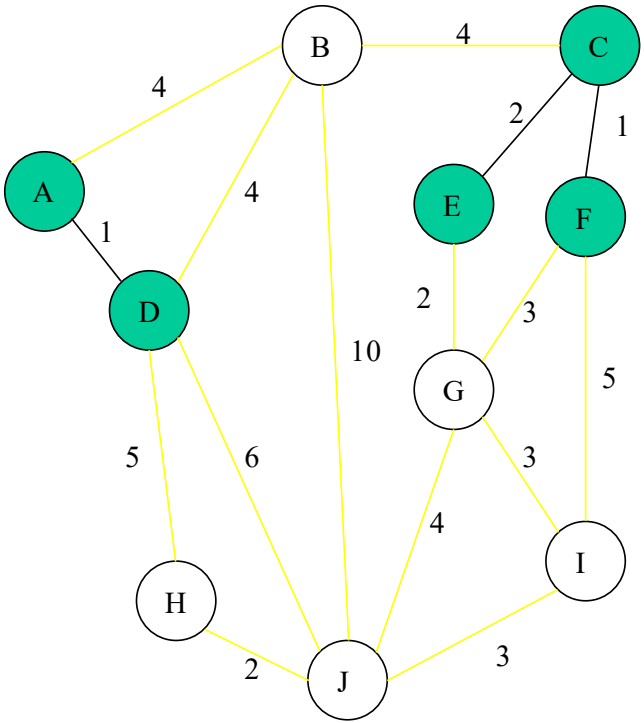
Add Edge



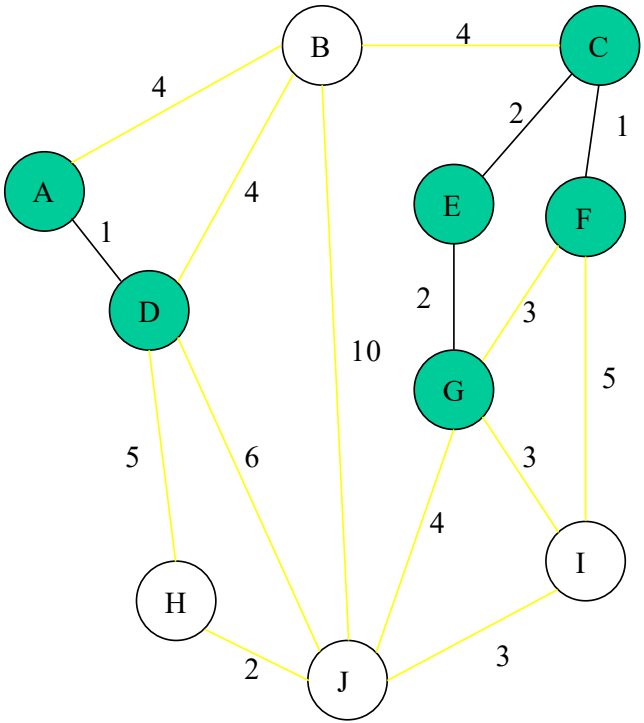
Add Edge



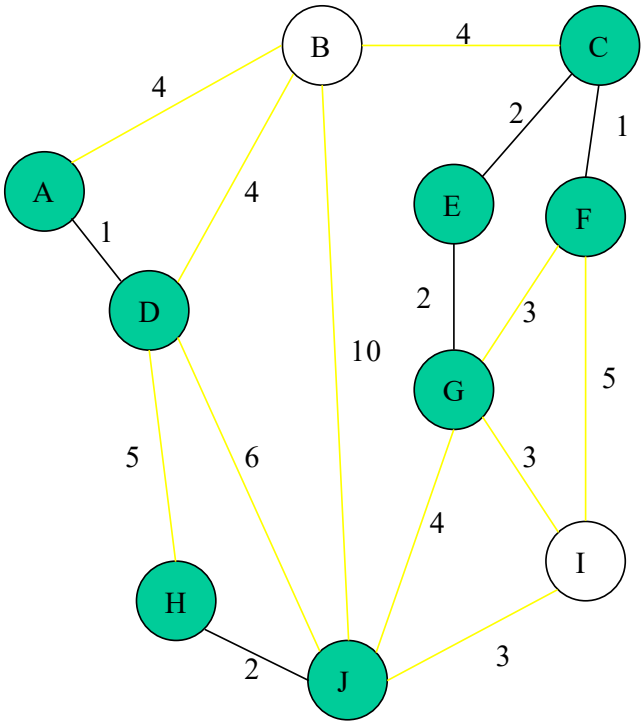
Add Edge



Add Edge

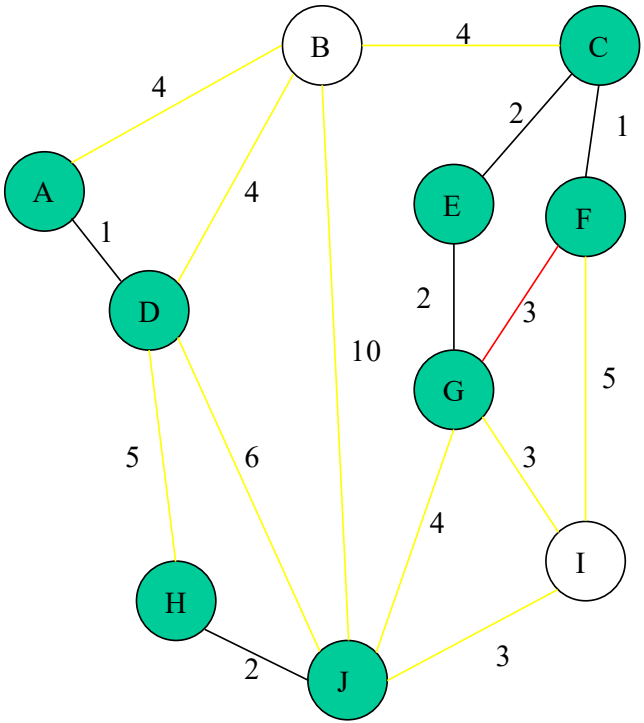


Add Edge

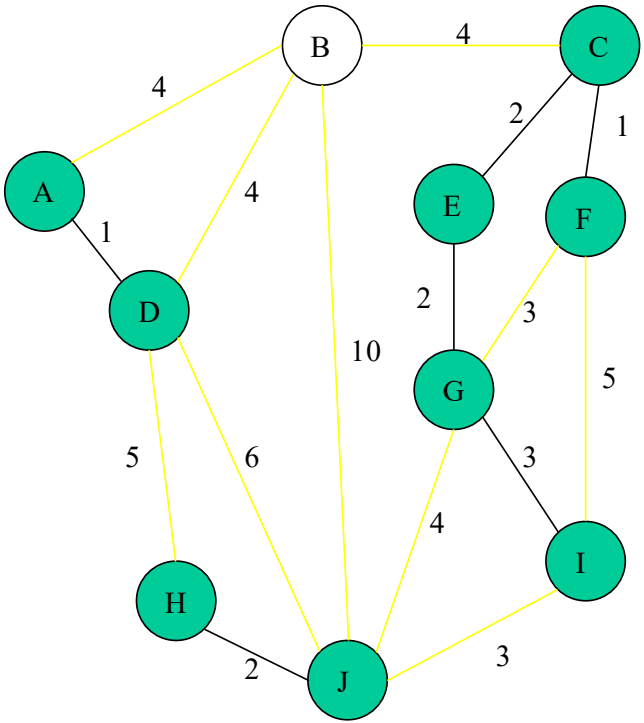


Cycle

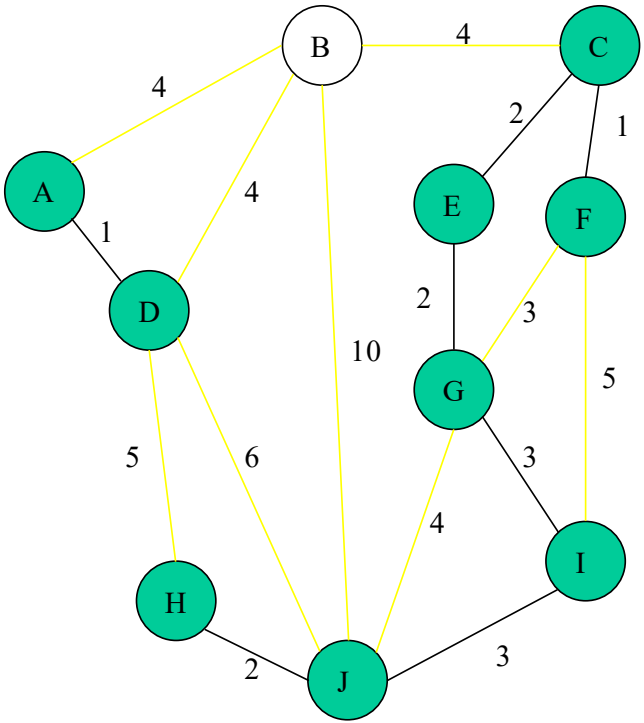
Don’t Add Edge



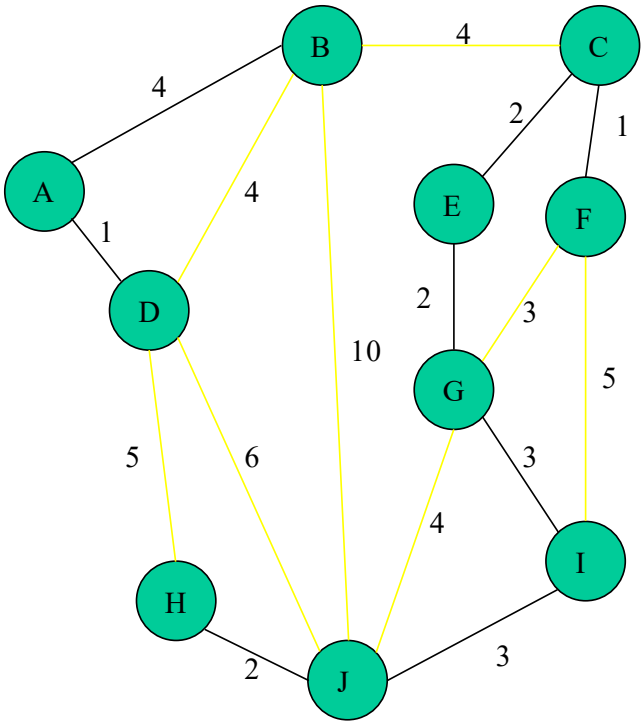
Add Edge



Add Edge

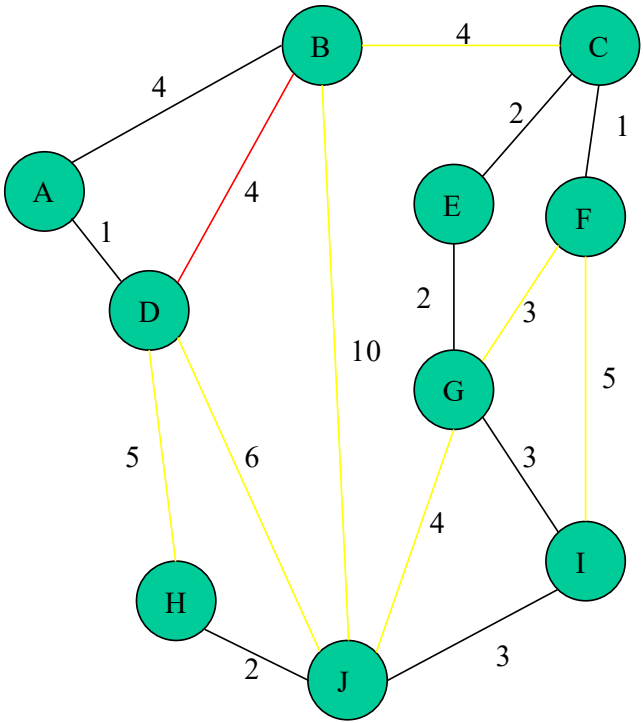


Add Edge

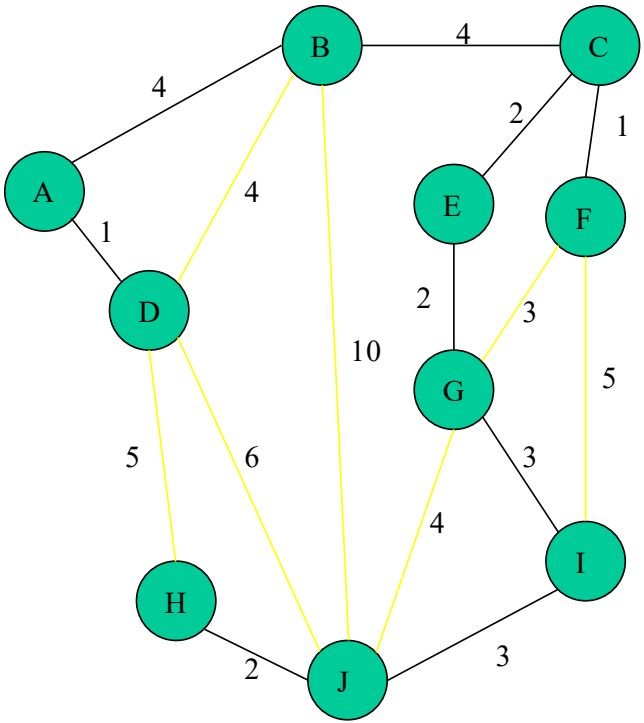


Cycle

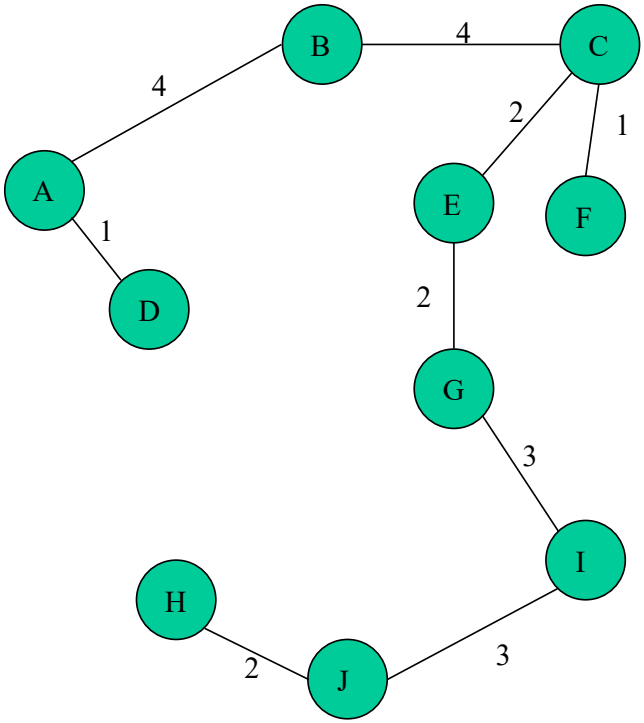
Don’t Add Edge



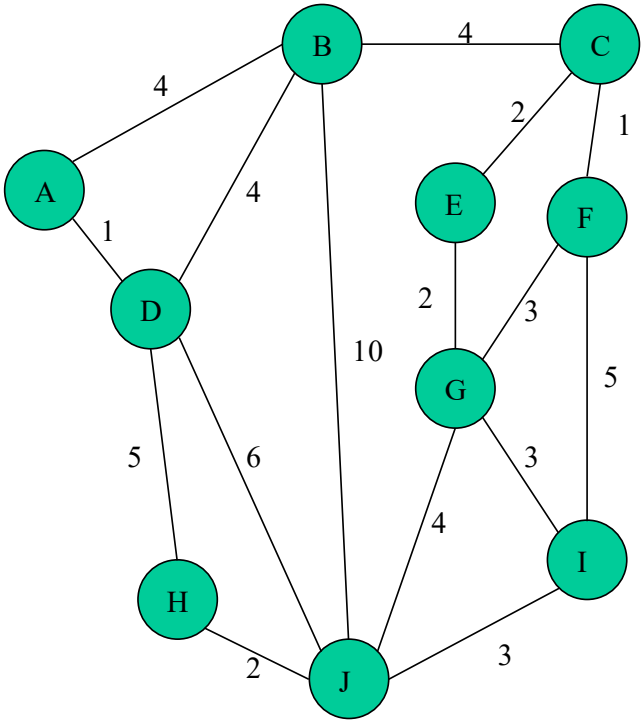
Add Edge



Minimum Spanning Tree



Complete Graph



Analysis of Kruskal's Algorithm

Running Time = O(m log n) (m = edges, n = nodes)

Testing if an edge creates a cycle can be slow

It usually only has to check a small fraction of the edges, but in some cases (like if there was a vertex connected to the graph by only one edge and it was the longest edge) it would have to check all the edges.

This algorithm works best, of course, if the number of edges is kept to a minimum.

Prim's Algorithm

Start from any node, choose the cheapest edge then the next cheapest one

This algorithm starts with one node. It then, one by one, adds a node that is unconnected to the new graph, each time selecting the node whose connecting edge has the smallest weight out of the available nodes' connecting edges.

In the worst case, it's exactly like Krugal $\Rightarrow O(m \log n)$

or $O(e \log v)$ (edges, nodes/vertices)

The steps are:

1. The new graph is constructed - with one node from the old graph.
2. While new graph has fewer than n nodes,
    1. Find the node from the old graph with the smallest connecting edge to the new graph,
    2. Add it to the new graph

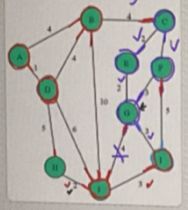
Every step will have joined one node, so that at the end we will have one graph with all the nodes and it will be a minimum spanning tree of the original graph.

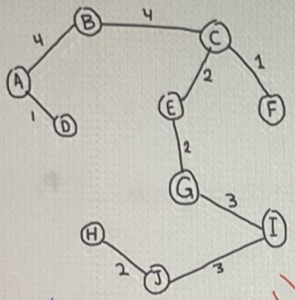
Course: Design and Analysis of | Untitled – FigJam

figma.com/board/LFFuBRWDnUrF9oj6GKsX1F/Untitled?node-id=0-1&p=f&t=elxlHwhHe6q8l2jq-0

Untitled Free

Complete Graph



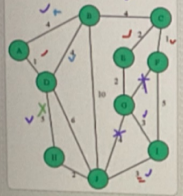


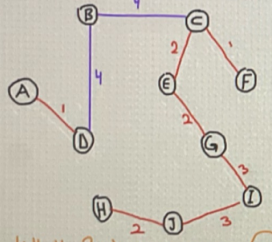
Prim

1+4+4+2+1+2+3+3+2

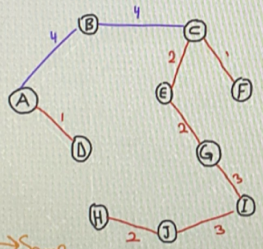
= 22

Complete Graph





1+4+4+2+1+2+3+3+2 = (22) → Samo



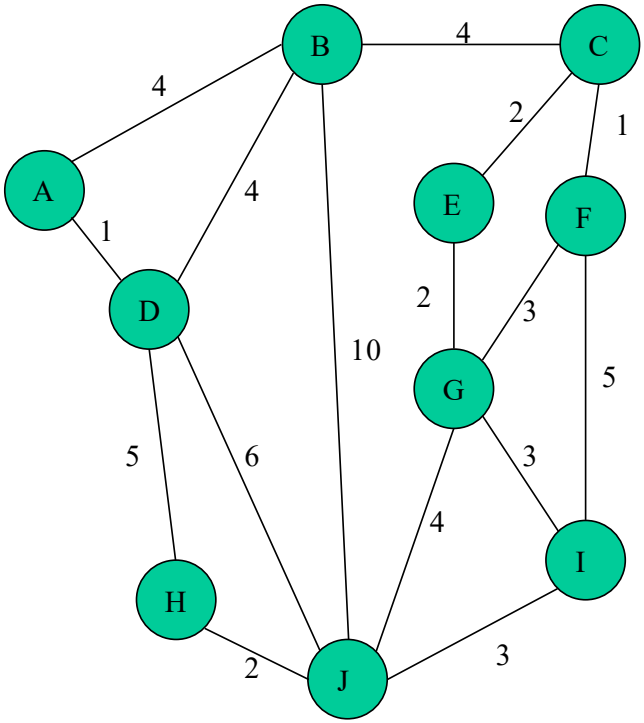
Kruskal

EUR/GBP

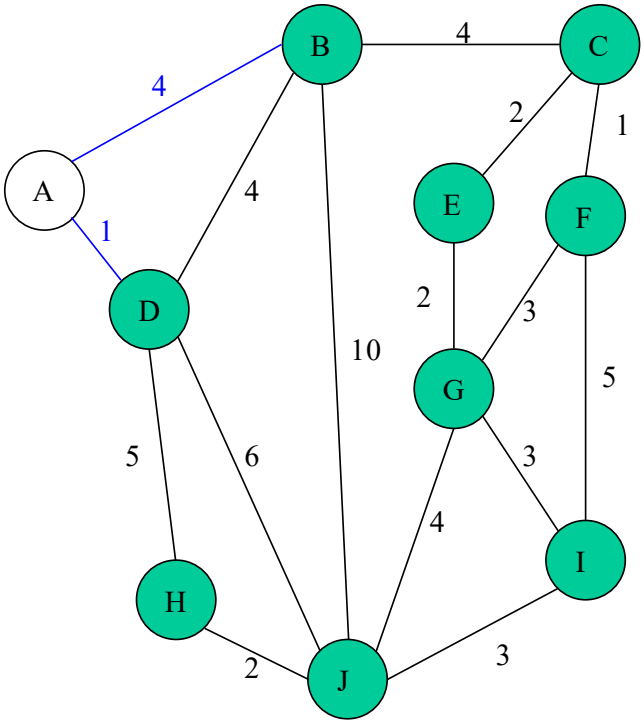
Search

ENG

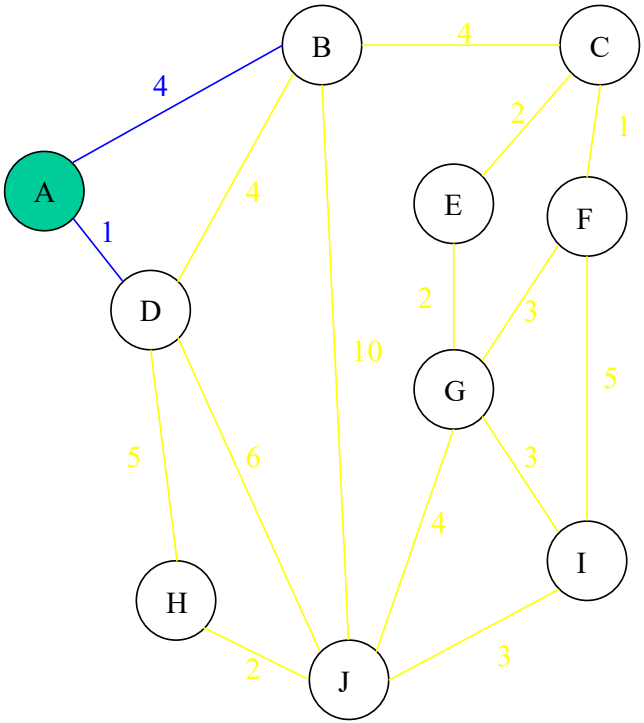
Complete Graph



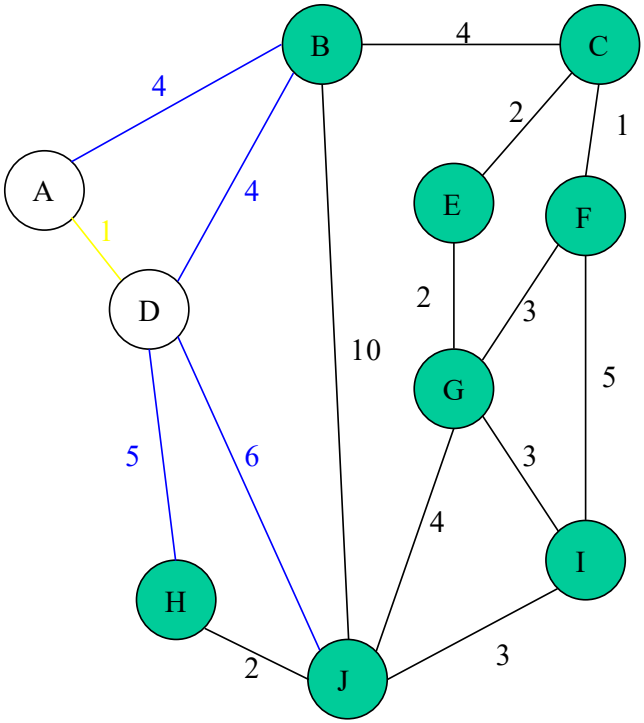
Old Graph



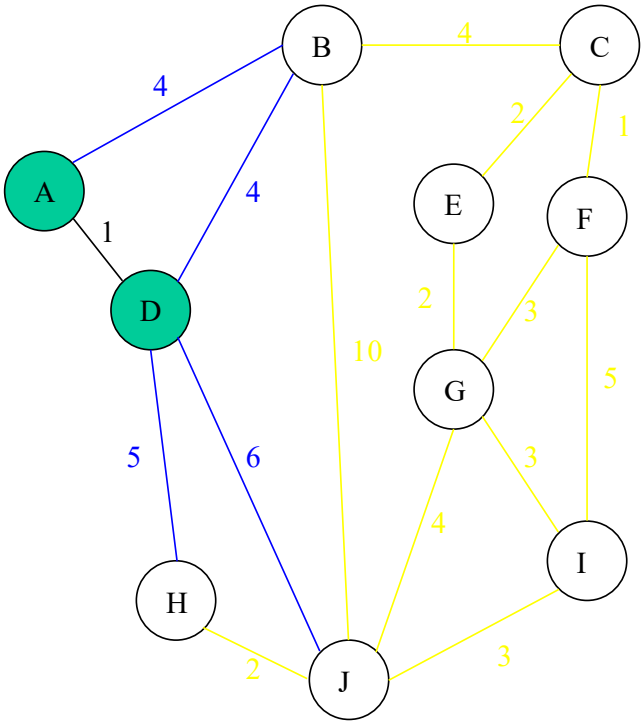
New Graph



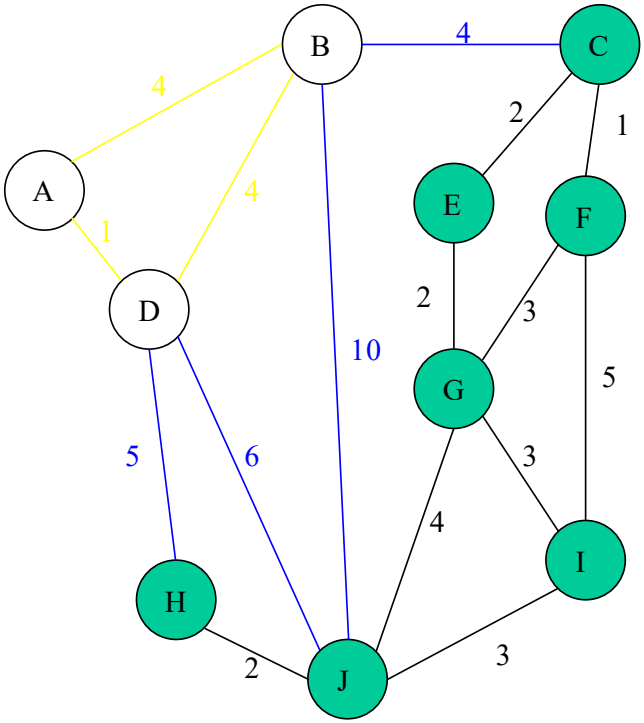
Old Graph



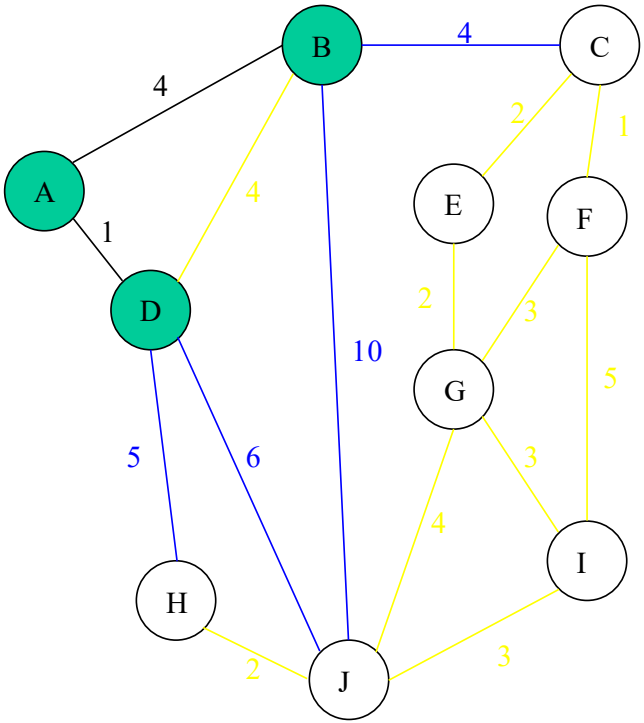
New Graph



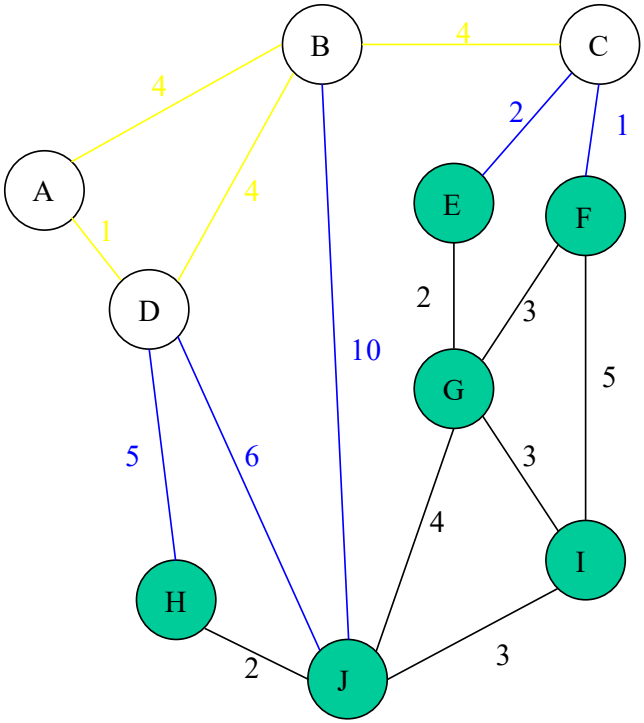
Old Graph



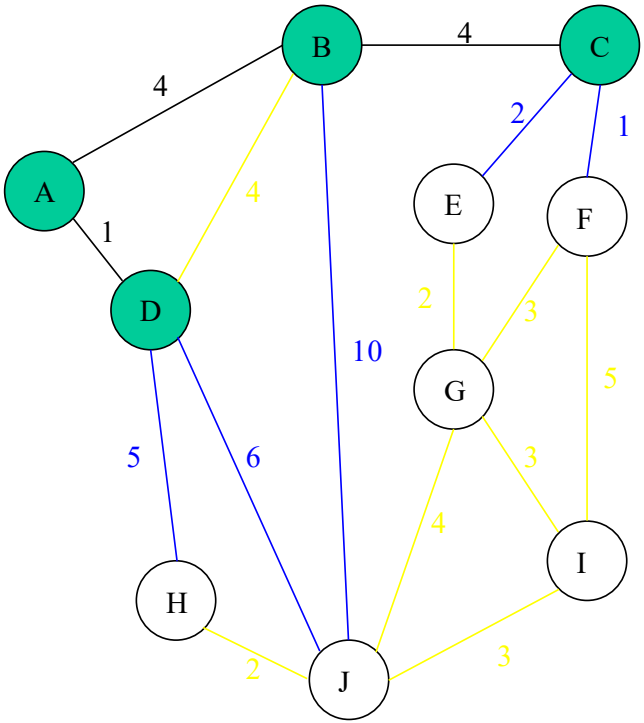
New Graph



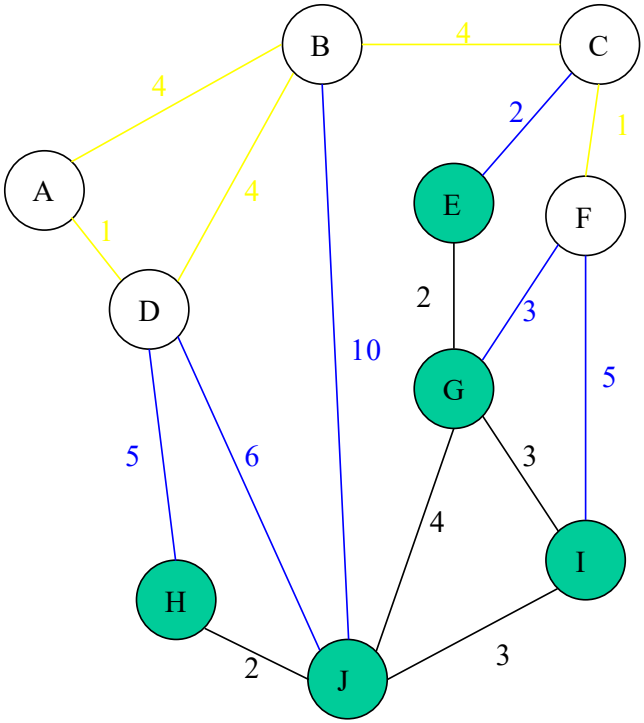
Old Graph



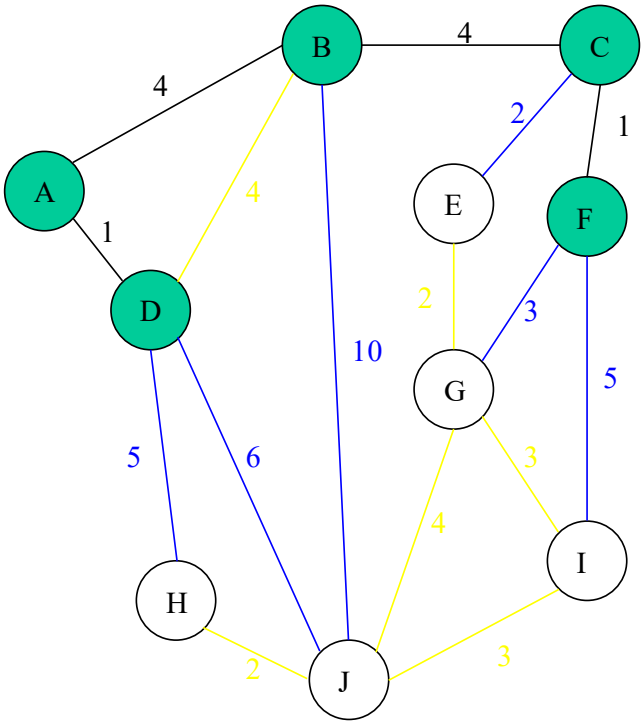
New Graph



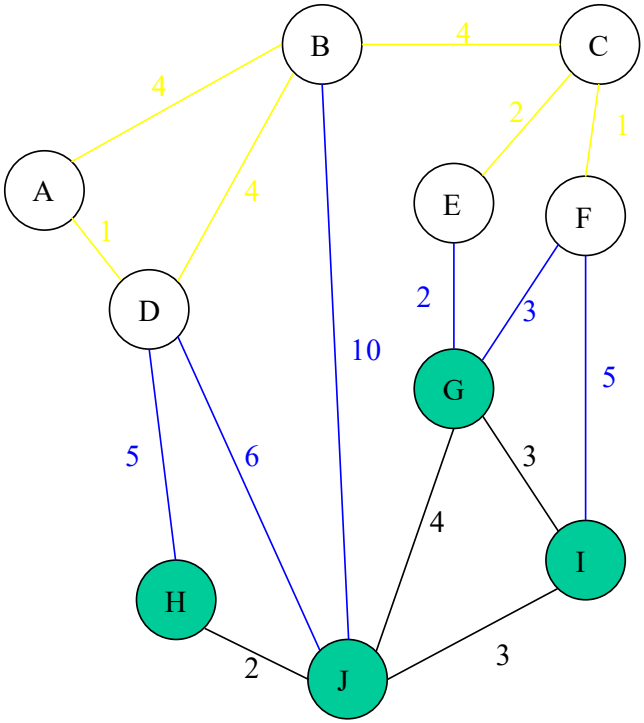
Old Graph



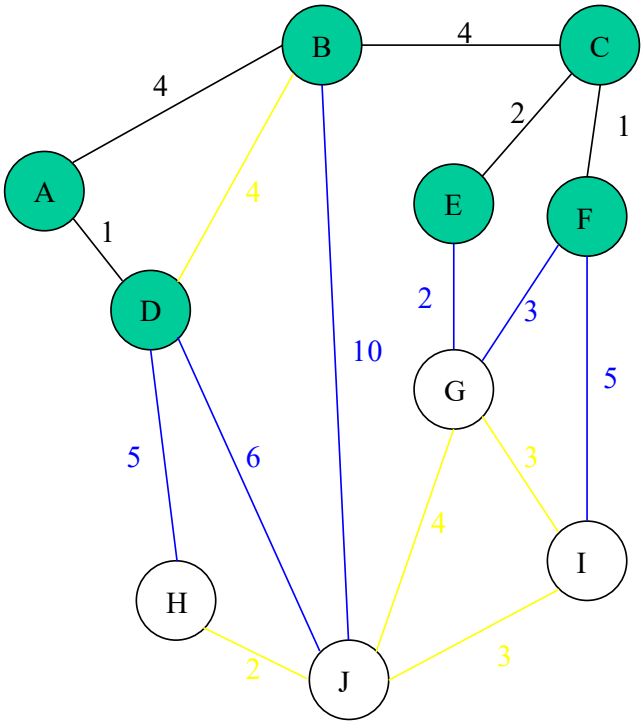
New Graph



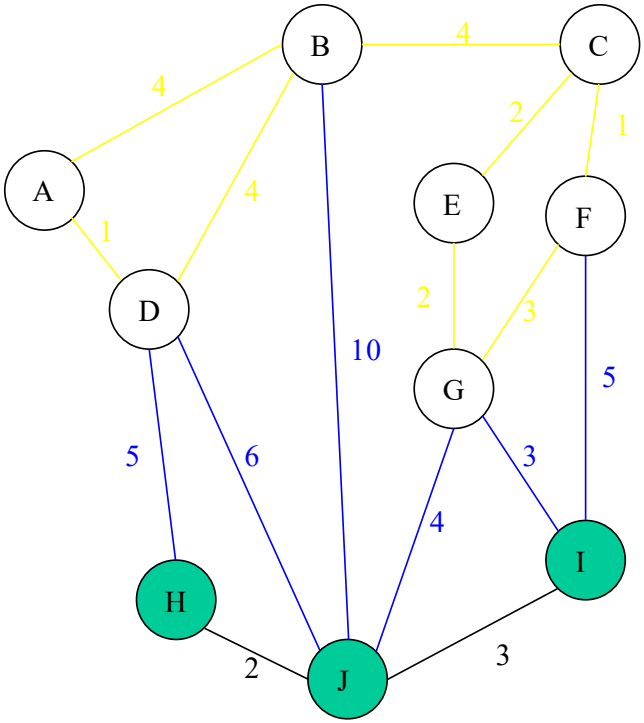
Old Graph



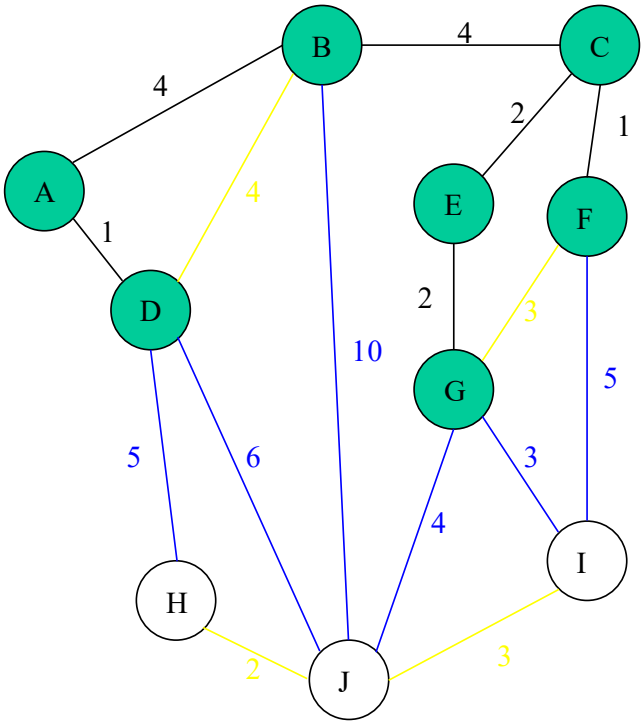
New Graph



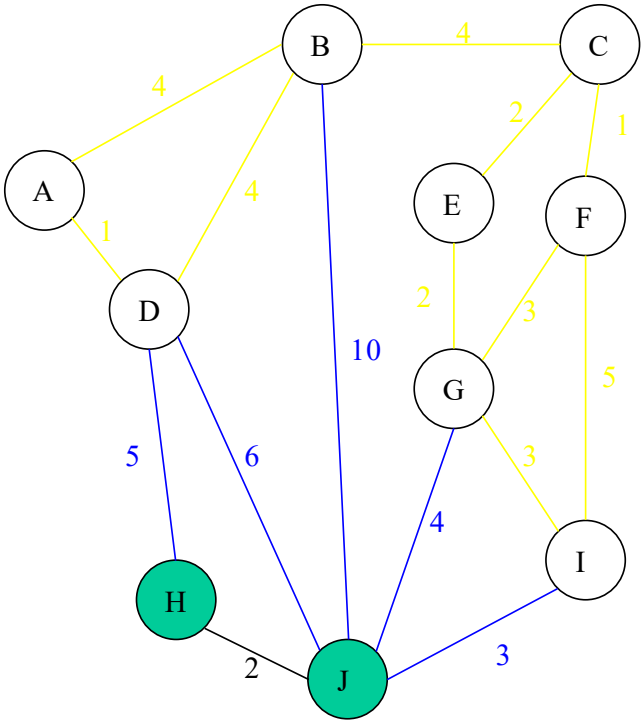
Old Graph



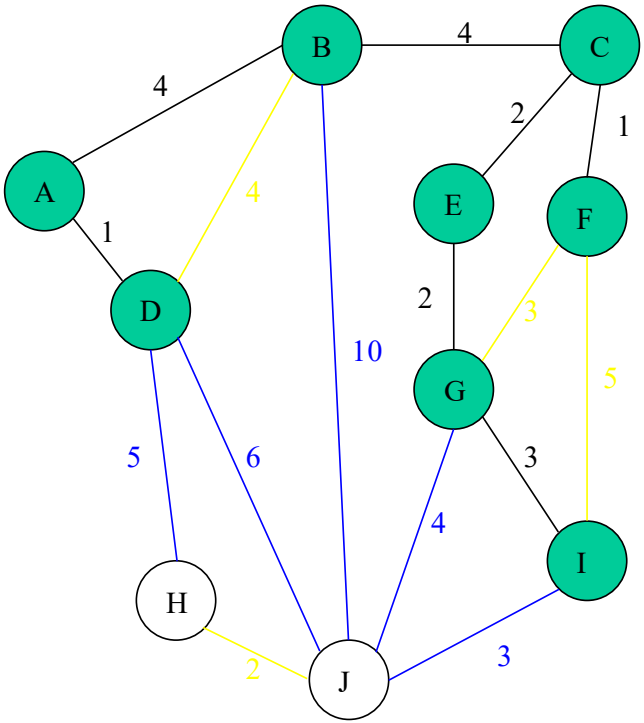
New Graph



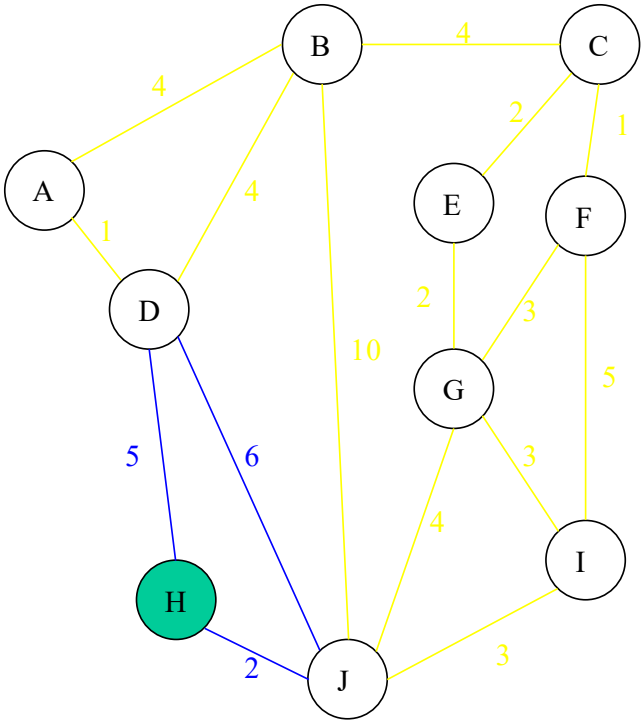
Old Graph



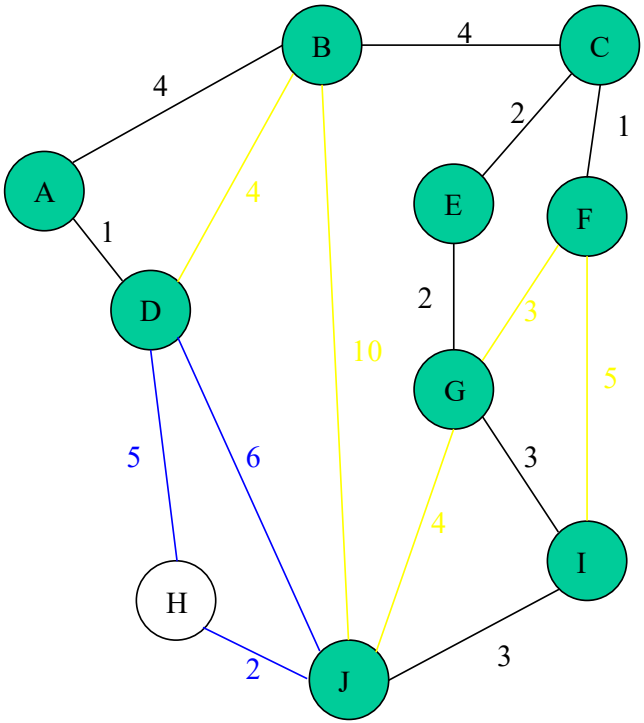
New Graph



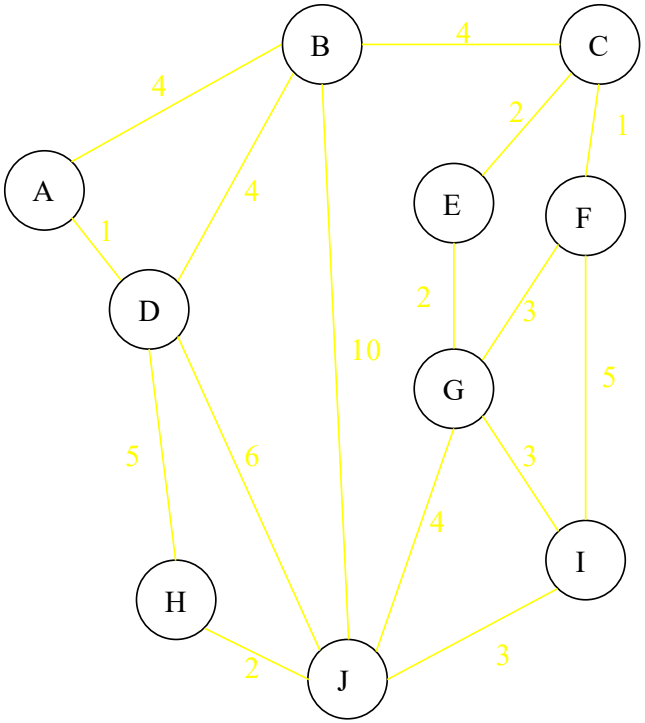
Old Graph



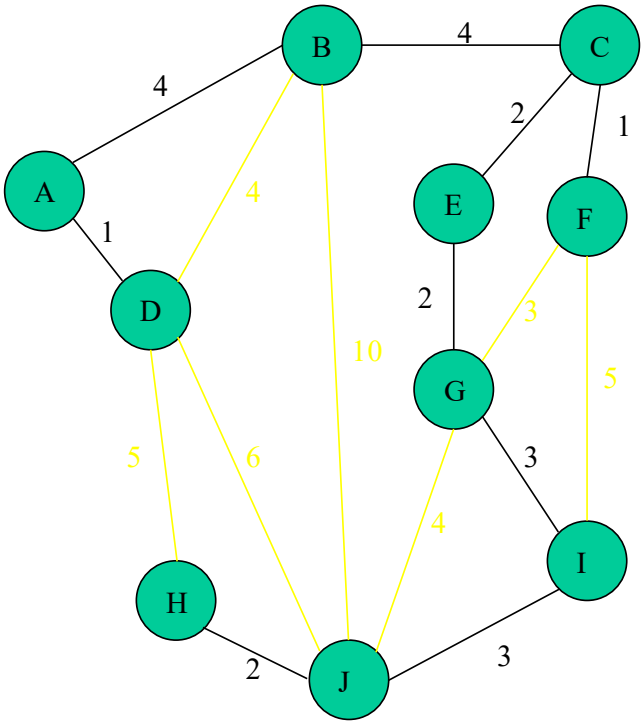
New Graph



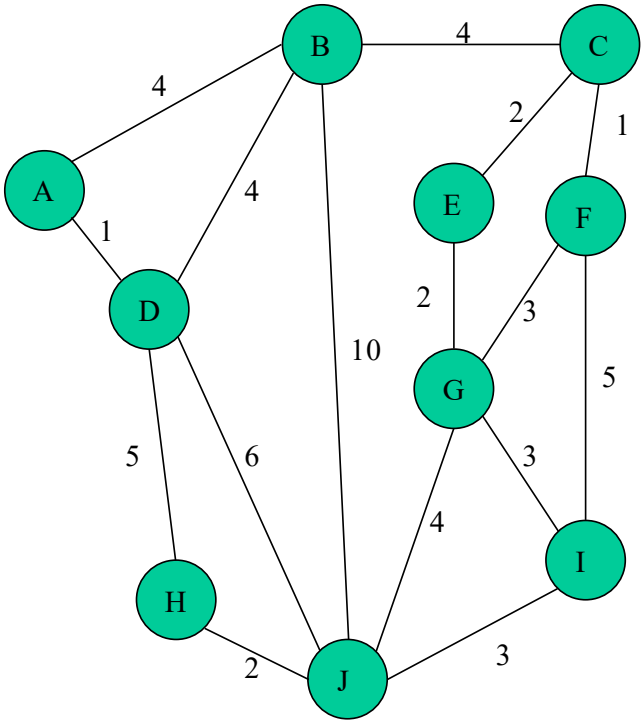
Old Graph



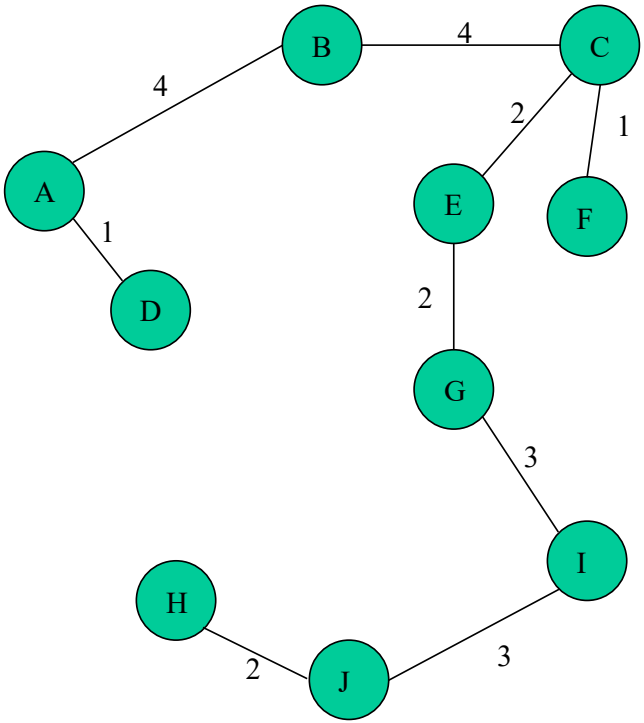
New Graph



Complete Graph



Minimum Spanning Tree



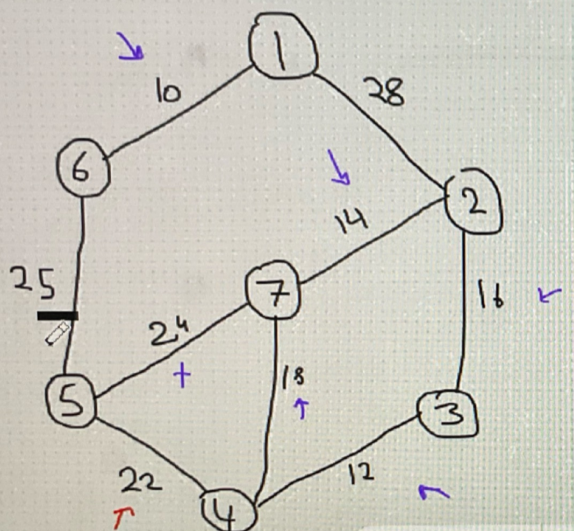
Conclusion

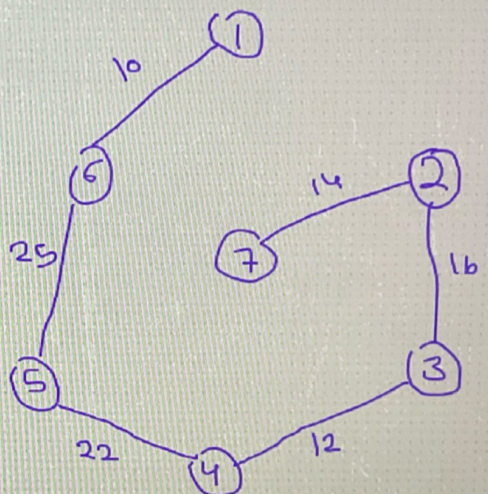
Kruskal's has better running times if the number of edges is low, while Prim's has a better running time if both the number of edges and the number of nodes are low.

So, the best algorithm depends on the graph

MST

Kruskal

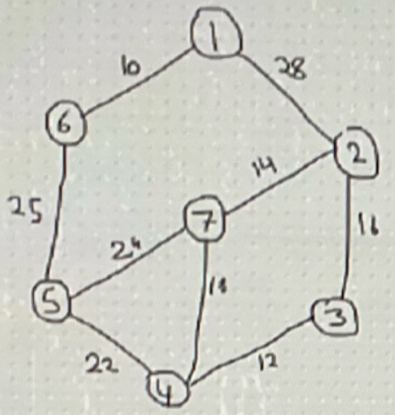




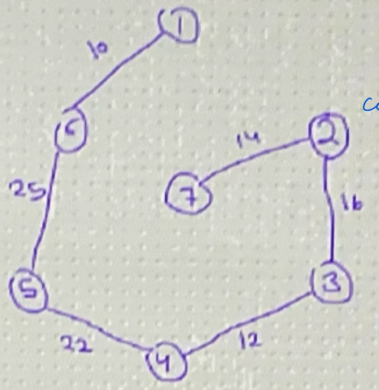
☆ No edges are repeated, each number is different so this graph only has 1 minimum spanning tree

So trees with different costs will never have more than one minimum spanning tree

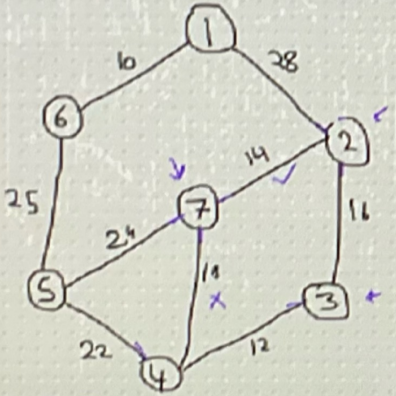
MST



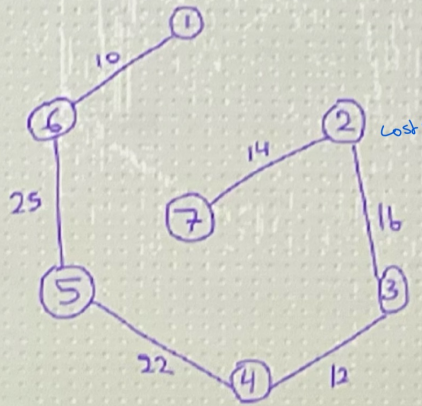
kruskal



cost: 10+25+22+12+16+14 = 99



Prime



cost: 10+25+22+12+16+14 = 99